

팍리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



9일차

01 Sensing Zone 과 HPC Zone

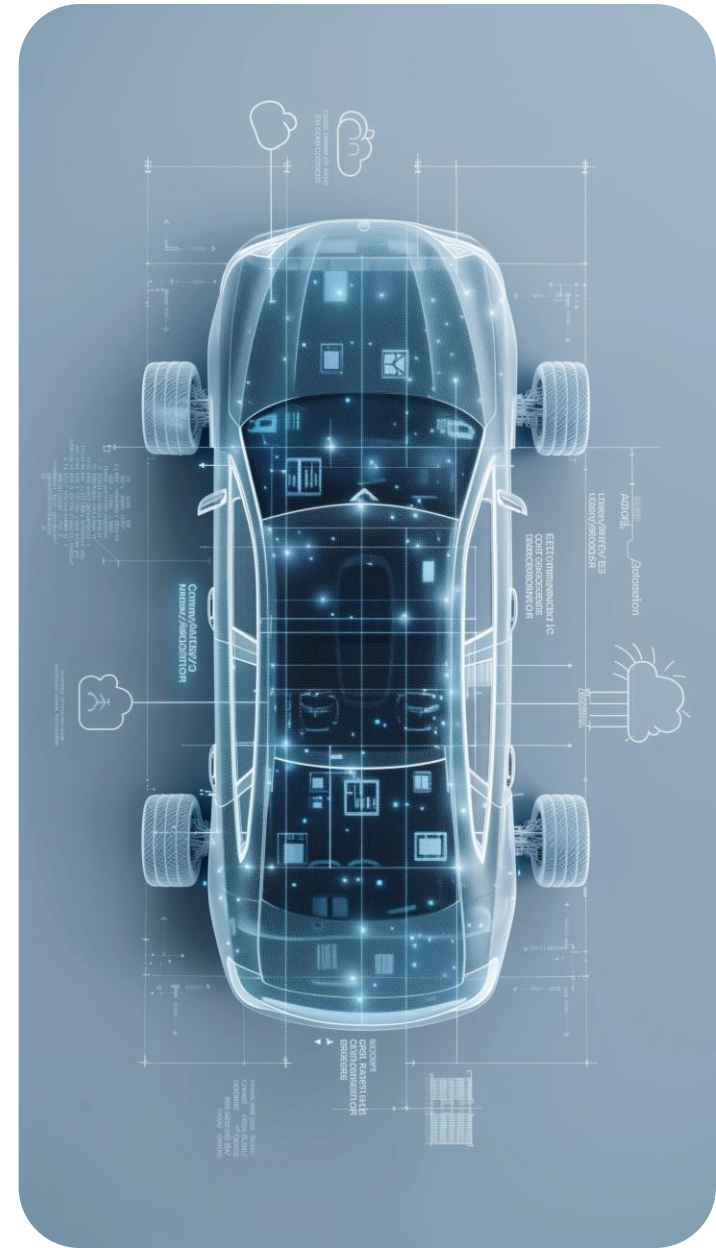
Telechips TOPST



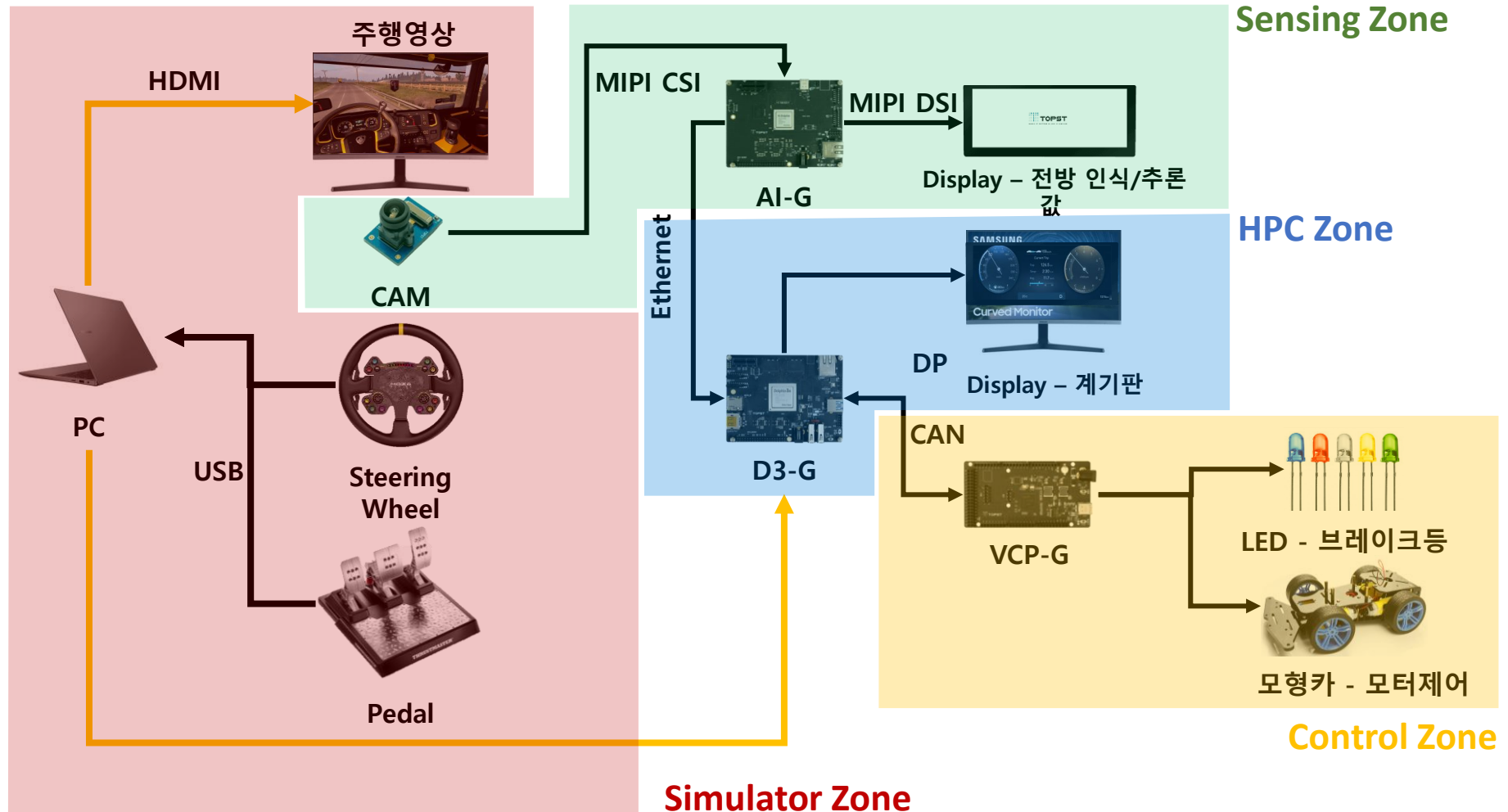
목차

Table of Contents

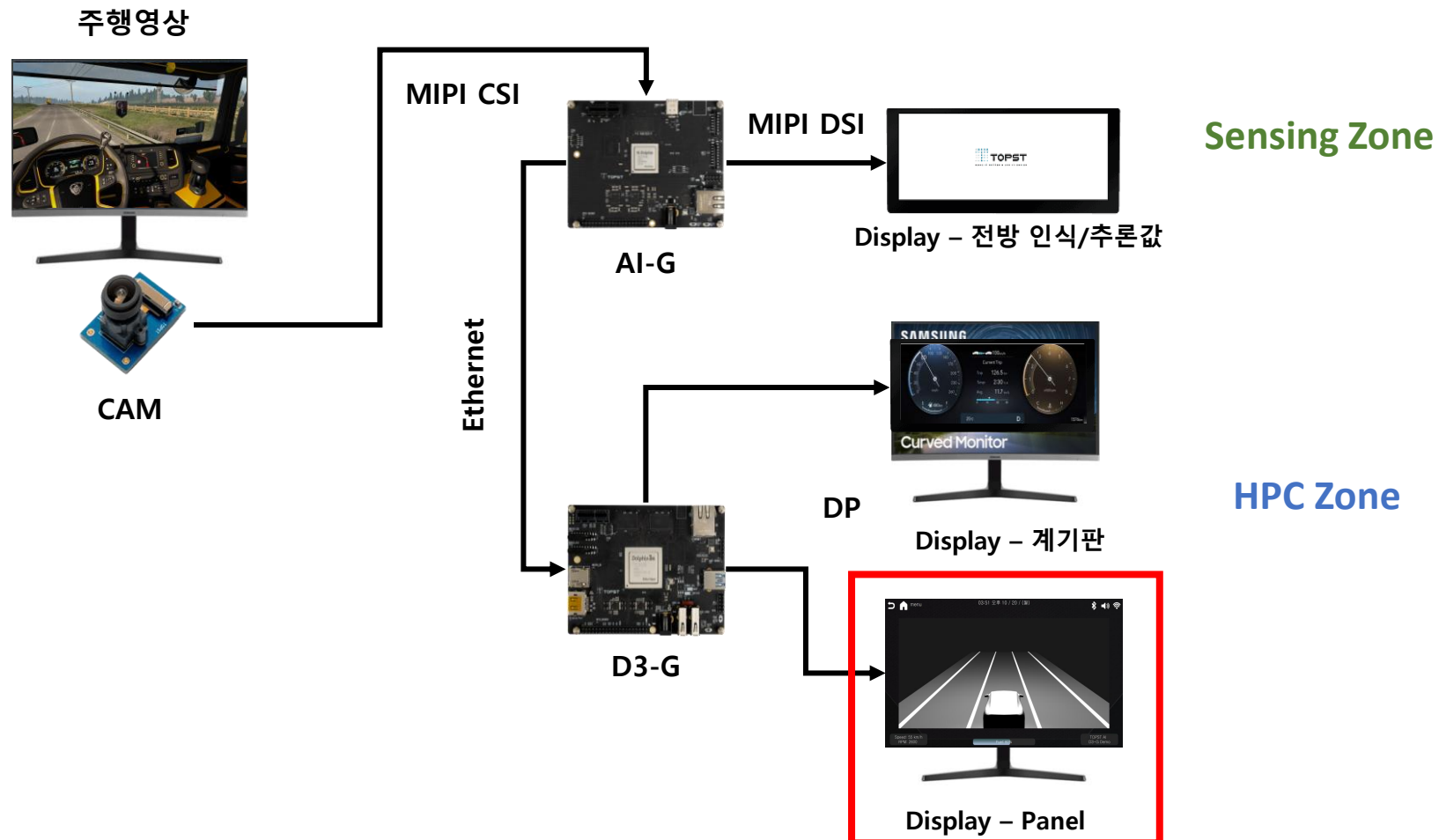
Sensing + HPC Zone 이해	01
AI-G의 postprocess	02
D3-G에 Panel App 소개	03



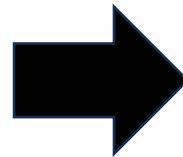
Zonal 기반 차량 인터랙션 시스템-전체 시나리오



Sensing Zone + HPC Zone 이해

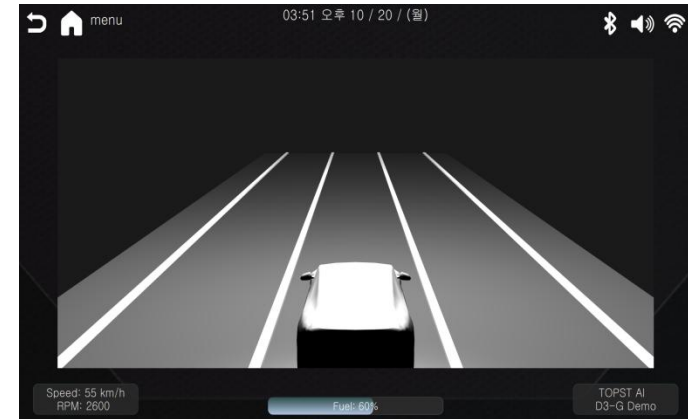
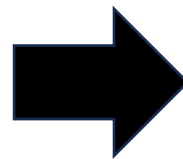
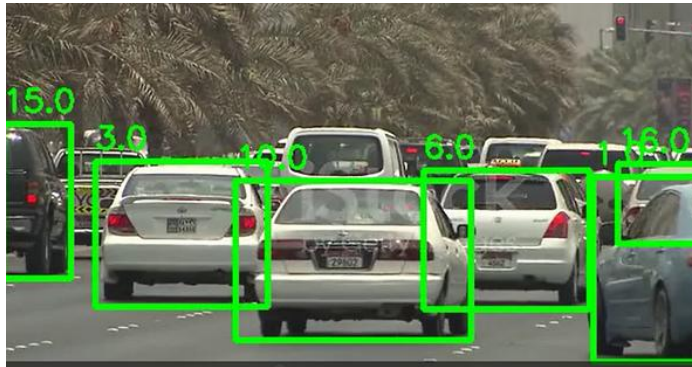


Sensing Zone + HPC Zone 이해



- AI-G 카메라를 통해 주행 영상 추론
- YOLOv8 모델을 통해 차량을 인식
- 인식한 차량의 좌표 값을 통해 차량의 위치 파악

Sensing Zone + HPC Zone 이해



- 이더넷 통신을 통해 D3-G의 QT Panel앱으로 전송
- QT앱에서 메시지를 수신
- 수신된 메시지를 반영하여 Panel 앱에 차량 표시

AI-G의 postprocess 코드 리뷰

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in servaddr = {0};
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(6000);

bind(sockfd, (struct sockaddr*)&servaddr,
sizeof(servaddr));
listen(sockfd, 1);
int connfd = accept(sockfd, NULL, NULL);
```

- AI-G의 tc-nn-app 소스에서 socket 열기
- 대기모드 상태로 진입하여 qt 앱에서 접속을 위해 대기 상태 진입

AI-G의 postprocess 코드 리뷰

```

    boxImageWidth  = pNeuralNetwork-
>resultObj.obj[index].img_w;
    boxImageHeight = pNeuralNetwork-
>resultObj.obj[index].img_h;

    // 원본 bbox (모델 좌표계)
    const int xmin = (int)(pNeuralNetwork-
>resultObj.obj[index].x_min + 0.5);
    const int ymin = (int)(pNeuralNetwork-
>resultObj.obj[index].y_min + 0.5);
    const int xmax = (int)(pNeuralNetwork-
>resultObj.obj[index].x_max + 0.5);
    const int ymax = (int)(pNeuralNetwork-
>resultObj.obj[index].y_max + 0.5);

    const int w = (xmax - xmin > 0) ? (xmax - xmin) : 0;
    const int h = (ymax - ymin > 0) ? (ymax - ymin) : 0;

    // 분류
    const char* lane_msg = classify_lane(xmin, ymin, w, h,
boxImageWidth, boxImageHeight);

```

```

const int cx = (xmin + xmax) / 2;
const int cy = (ymin + ymax) / 2;

// 3) 영역 분할
const int cx1 = OUT_W / 3;
const int cx2 = (OUT_W * 2) / 3;
const int cy1 = OUT_H / 3;
const int cy2 = (OUT_H * 2) / 3;

if (cx >= cx1 && cx <= cx2 && cy >= cy1 && cy <= cy2)
{
    return "object_detected_lane_5"; // 중앙
} else if (cx >= OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_4"; // 우측 상단
} else if (cx >= OUT_W / 2 && cy >= OUT_H / 2) {
    return "object_detected_lane_1"; // 우측 하단
} else if (cx < OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_3"; // 좌측 상단
} else {
    return "object_detected_lane_2"; // 좌측 하단
}
}

```

- 모델의 타입이 object detection 모델일 경우에 해당하는 경우에 후처리 기능 추가
- 객체를 인식한 좌표에 대한 정보를 추출

AI-G의 postprocess 코드 리뷰

```

    boxImageWidth  = pNeuralNetwork-
>resultObj.obj[index].img_w;
    boxImageHeight = pNeuralNetwork-
>resultObj.obj[index].img_h;

    // 원본 bbox (모델 좌표계)
    const int xmin = (int)(pNeuralNetwork-
>resultObj.obj[index].x_min + 0.5);
    const int ymin = (int)(pNeuralNetwork-
>resultObj.obj[index].y_min + 0.5);
    const int xmax = (int)(pNeuralNetwork-
>resultObj.obj[index].x_max + 0.5);
    const int ymax = (int)(pNeuralNetwork-
>resultObj.obj[index].y_max + 0.5);

    const int w = (xmax - xmin > 0) ? (xmax - xmin) : 0;
    const int h = (ymax - ymin > 0) ? (ymax - ymin) : 0;

    // 분류
    const char* lane_msg = classify_lane(xmin, ymin, w, h,
boxImageWidth, boxImageHeight);

```

```

const int cx = (xmin + xmax) / 2;
const int cy = (ymin + ymax) / 2;

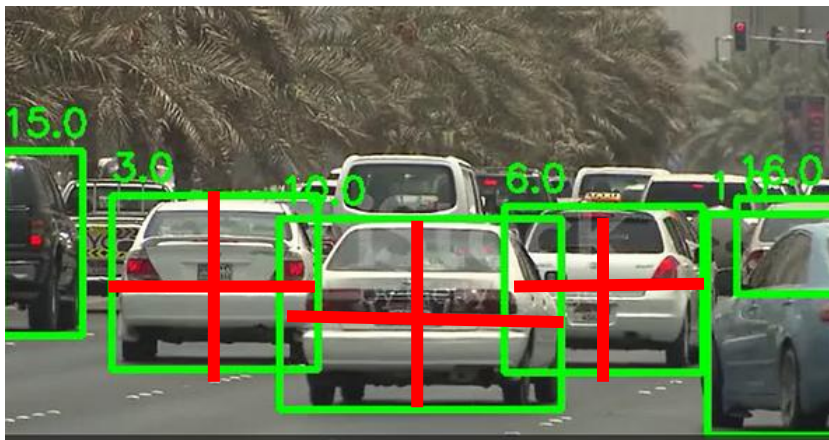
// 3) 영역 분할
const int cx1 = OUT_W / 3;
const int cx2 = (OUT_W * 2) / 3;
const int cy1 = OUT_H / 3;
const int cy2 = (OUT_H * 2) / 3;

if (cx >= cx1 && cx <= cx2 && cy >= cy1 && cy <= cy2)
{
    return "object_detected_lane_5"; // 중앙
} else if (cx >= OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_4"; // 우측 상단
} else if (cx >= OUT_W / 2 && cy >= OUT_H / 2) {
    return "object_detected_lane_1"; // 우측 하단
} else if (cx < OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_3"; // 좌측 상단
} else {
    return "object_detected_lane_2"; // 좌측 하단
}
}

```

- 추출한 정보를 바탕으로 중심 좌표 계산
- 임의로 인식한 화면의 경계를 나눠 구역에 따라 다른 메시지를 송신

AI-G의 postprocess 코드 리뷰



중심 좌표 계산

```
const int cx = (xmin + xmax) / 2;
const int cy = (ymin + ymax) / 2;

// 3) 영역 분할
const int cx1 = OUT_W / 3;
const int cx2 = (OUT_W * 2) / 3;
const int cy1 = OUT_H / 3;
const int cy2 = (OUT_H * 2) / 3;

if (cx >= cx1 && cx <= cx2 && cy >= cy1 && cy <= cy2)
{
    return "object_detected_lane_5"; // 중앙
} else if (cx >= OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_4"; // 우측 상단
} else if (cx >= OUT_W / 2 && cy >= OUT_H / 2) {
    return "object_detected_lane_1"; // 우측 하단
} else if (cx < OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_3"; // 좌측 상단
} else {
    return "object_detected_lane_2"; // 좌측 하단
}
}
```

- 추출한 정보를 바탕으로 중심 좌표 계산
- 임의로 인식한 화면의 경계를 나눠 구역에 따라 다른 메시지를 송신

AI-G의 postprocess 코드 리뷰



```
const int cx = (xmin + xmax) / 2;
const int cy = (ymin + ymax) / 2;

// 3) 영역 분할
const int cx1 = OUT_W / 3;
const int cx2 = (OUT_W * 2) / 3;
const int cy1 = OUT_H / 3;
const int cy2 = (OUT_H * 2) / 3;

if (cx >= cx1 && cx <= cx2 && cy >= cy1 && cy <= cy2)
{
    return "object_detected_lane_5"; // 중앙
} else if (cx >= OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_4"; // 우측 상단
} else if (cx >= OUT_W / 2 && cy >= OUT_H / 2) {
    return "object_detected_lane_1"; // 우측 하단
} else if (cx < OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_3"; // 좌측 상단
} else {
    return "object_detected_lane_2"; // 좌측 하단
}
}
```

- 추출한 정보를 바탕으로 중심 좌표 계산
- 임의로 인식한 화면의 경계를 나눠 구역에 따라 다른 메시지를 송신

AI-G의 postprocess 코드 리뷰



```
const int cx = (xmin + xmax) / 2;
const int cy = (ymin + ymax) / 2;

// 3) 영역 분할
const int cx1 = OUT_W / 3;
const int cx2 = (OUT_W * 2) / 3;
const int cy1 = OUT_H / 3;
const int cy2 = (OUT_H * 2) / 3;

if (cx >= cx1 && cx <= cx2 && cy >= cy1 && cy <= cy2)
{
    return "object_detected_lane_5"; // 중앙
} else if (cx >= OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_4"; // 우측 상단
} else if (cx >= OUT_W / 2 && cy >= OUT_H / 2) {
    return "object_detected_lane_1"; // 우측 하단
} else if (cx < OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_3"; // 좌측 상단
} else {
    return "object_detected_lane_2"; // 좌측 하단
}
}
```

- 중앙에 해당할 경우 object_detected_lane_5 송신
- 나머지는 각각 lane_* 송신

AI-G의 postprocess 코드 리뷰



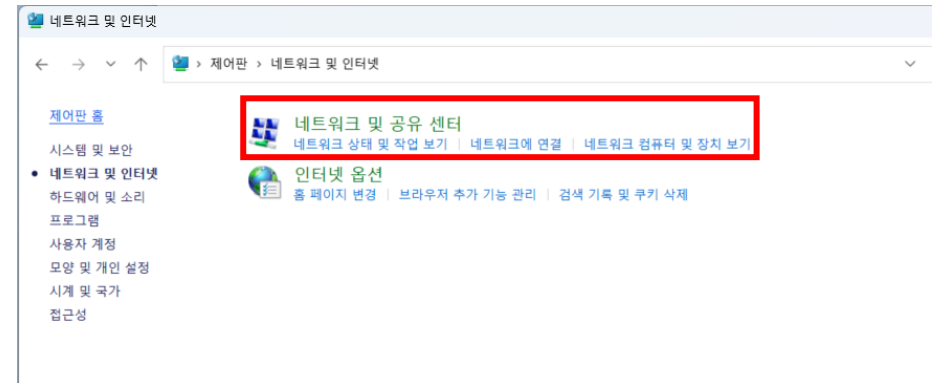
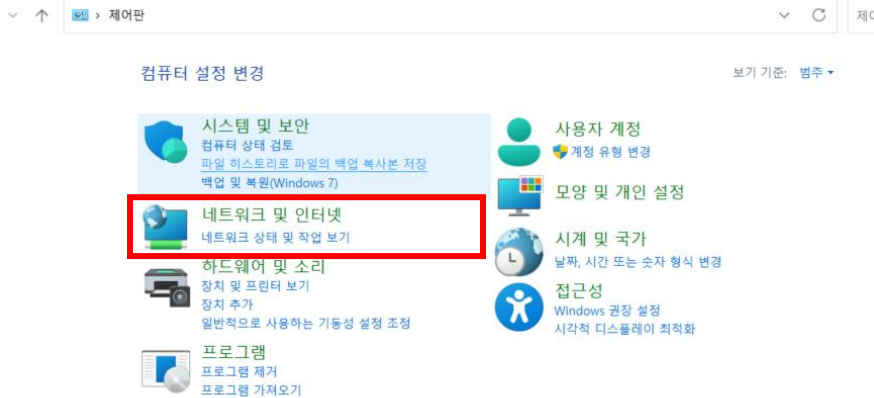
```
const int cx = (xmin + xmax) / 2;
const int cy = (ymin + ymax) / 2;

// 3) 영역 분할
const int cx1 = OUT_W / 3;
const int cx2 = (OUT_W * 2) / 3;
const int cy1 = OUT_H / 3;
const int cy2 = (OUT_H * 2) / 3;

if (cx >= cx1 && cx <= cx2 && cy >= cy1 && cy <= cy2)
{
    return "object_detected_lane_5"; // 중앙
} else if (cx >= OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_4"; // 우측 상단
} else if (cx >= OUT_W / 2 && cy >= OUT_H / 2) {
    return "object_detected_lane_1"; // 우측 하단
} else if (cx < OUT_W / 2 && cy < OUT_H / 2) {
    return "object_detected_lane_3"; // 좌측 상단
} else {
    return "object_detected_lane_2"; // 좌측 하단
}
}
```

- 중앙에 해당할 경우 object_detected_lane_5 송신
- 나머지는 각각 lane_* 송신

AI-G의 postprocess

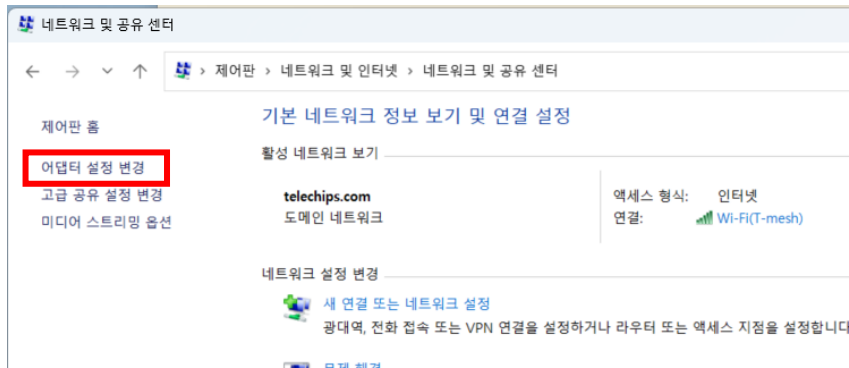


• Ethernet Set Up

▪ Host PC Network Configuration

➤ 제어판 -> 네트워크 및 인터넷 -> 네트워크 및 공유 센터 -> 어댑터 설정 변경

AI-G의 postprocess

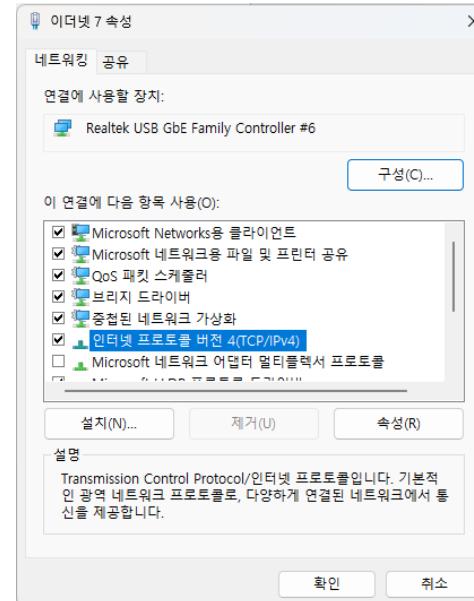


- **Ethernet Set Up**

- Host PC Network Configuration

- 제어판 -> 네트워크 및 인터넷 -> 네트워크 및 공유 센터 -> 어댑터 설정 변경

AI-G의 postprocess

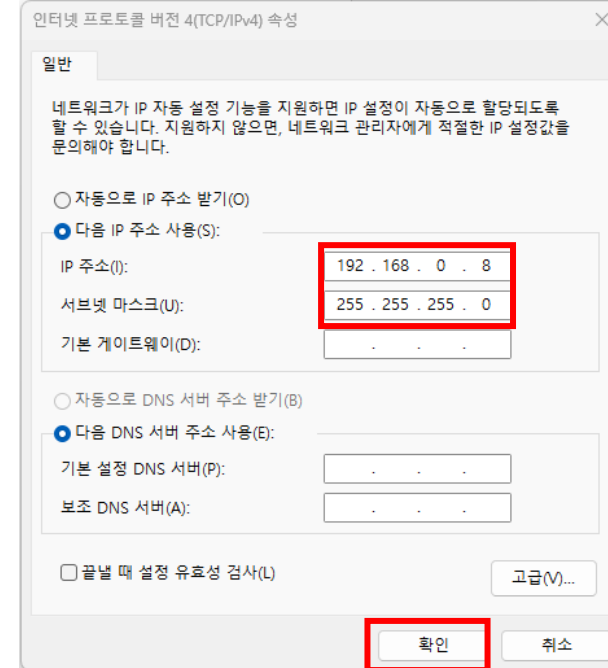
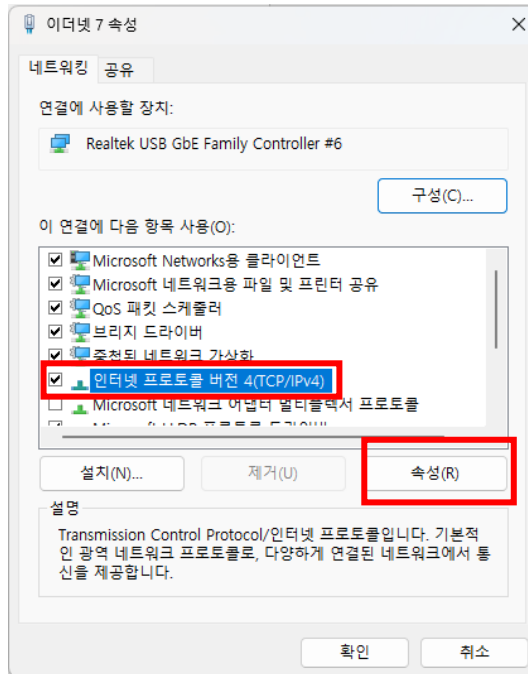


• Ethernet Set Up

▪ Host PC Network Configuration

- 이더넷 선택 -> 속성 -> 인터넷 프로토콜 버전4(TCP/IPv4) -> 속성

AI-G의 postprocess



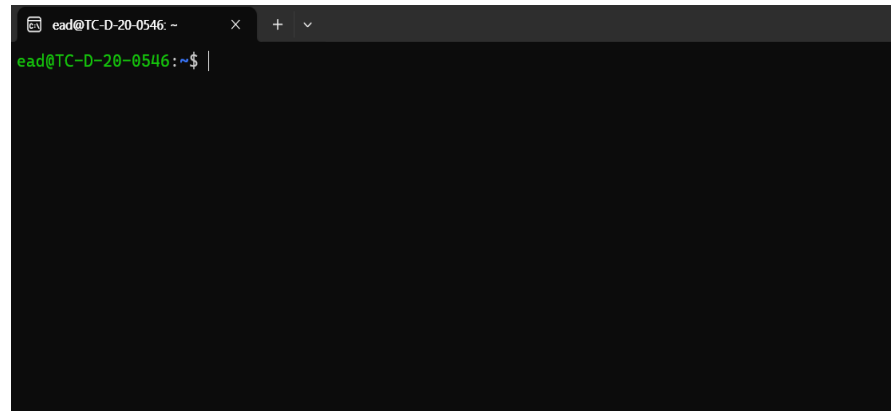
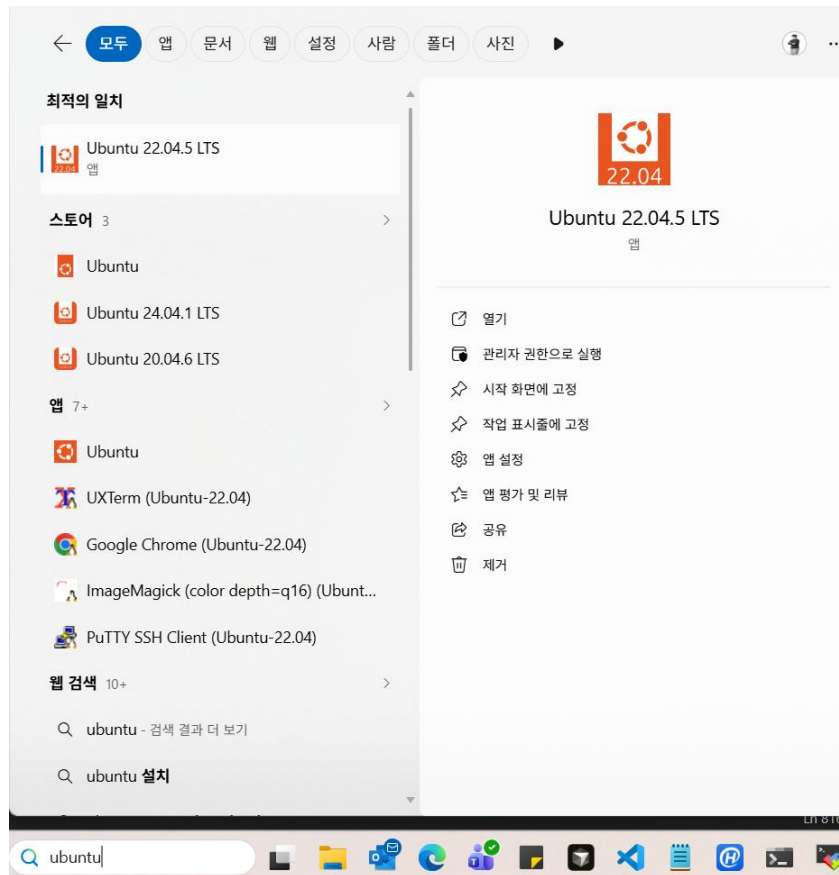
• Ethernet Set Up

▪ Host PC Network Configuration

- 다음 IP 주소 사용 -> IP 주소 : 192.168.0.8 / 서브넷 마스크 : 255.255.255.0 -> 확인

AI-G의 postprocess

- WSL Ubuntu 22.04 실행



AI-G의 postprocess

- Wsl ubuntu 접속
- 이전에 clone 한 Education 디렉터리로 이동

```
$ cd ~/Work/tc-nn-toolkit/input_networks/Education
```

```
ead@TC-D-20-0546:~$ cd tc-nn-toolkit/tc-nn-toolkit/input_networks/Education/  
ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit/input_networks/Education$ ls  
'ai-g app'  ai_model  'd3-g app'  LectureNote  qt  README.md
```

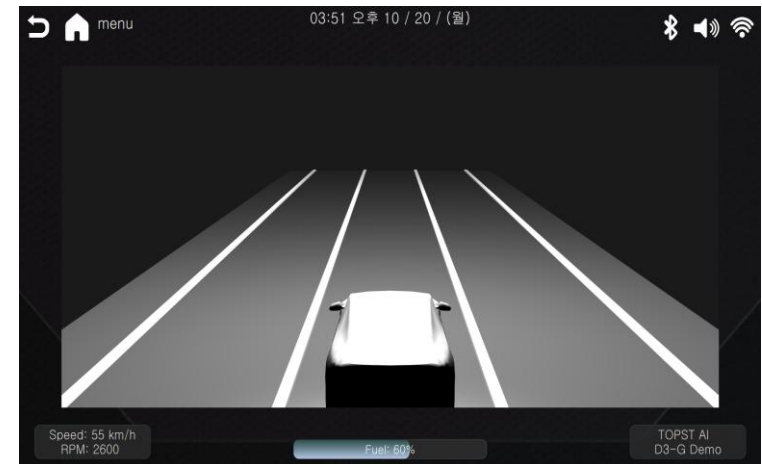
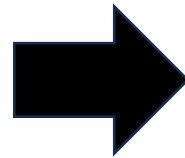
```
$ cd 'ai-g app'
```

```
ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit/input_networks/Education$ cd ai-g\ app/  
ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit/input_networks/Education/ai-g app$ ls  
demoapp  tcnnapp
```

사용할 앱 ai-g 보드로 전송 (ai-g보드 전원 인가 상태)

```
$ scp -r demoapp root@192.168.0.100:/home/root/
```

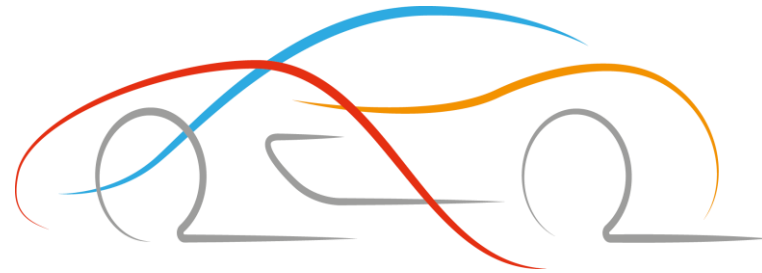
D3-G Panel APP 소개



- QT의 3D 기능과 모델링한 차량 모형을 활용해 제작한 Panel 앱
- 수신한 데이터를 기반으로 해당하는 위치에 차량 모형을 출력
- 차량이 인식될 때마다 차량 모형 출력

D3-G Panel APP 소개

- 추론 명령어 실행 (AI-G Board가 연결된 터미널에서 수행)
 - `chmod 755 demoapp`
 - `./demoapp -n yolov8s_quantized -N UFLD_quantized -p /dev/video2`



Thank You